Documentation for Assignment 3

Group members:
Manojna Reddy Kamaram: 2024334
Palak: 2024399
Shanthini Muralidhar: 2024520

A3: Creating a Mini Packet Analyzer
Github Link: https://github.com/shanthini11/CN_A3

Overall preview

Our mini packet analyzer's main goal here is to replicate the same doings of the Wireshark application

Functionalities to replicate includes:

- Captures network frames sent or received by the host, and numbers and timestamps each parsed packet.
- Shows the packet size, source/destination addresses, ports and protocol in a summary table.
- Decodes the IPv4, TCP and UDP headers manually from the raw bytes. Scapy is used only to capture the raw frames; it is not used to decode any header.
- Filters packets by protocol (TCP/UDP), source/destination IP and source/destination port.
- Displays a selected packet's IP header, TCP/UDP header, and payload in hexadecimal and text form

How do all the files work together?

1. The main python file first asks for the network interface, max packet counts and capture duration
2. Next, the capture file will use the sniff() function from Scapy to help capture frames and record bytes(packets) with a timestamp
3. The parser file converts each raw frame into a packet dictionary. It only recognizes Ethernet frames carrying IPv4 (with or without a VLAN tag); any other frame, such as ARP or IPv6, is skipped.
4. The menu helps the user choose whatever option they like to parse after running the main file and lets the user show all the parsed packets, filter packets, inspects the packet by a specific number or exit the program

How do we set it up in your personal systems?

MacOS/Linux System

Run these commands in the project folder:

python3 -m venv .venv

source .venv/bin/activate

python -m pip install -r requirements.txt

python test_parser.py

python test_filters.py

python test_display.py

python main.py

If packet capture fails because of permissions on macOS, try:
sudo .venv/bin/python main.py

Choose the interface connected to your network. On many Macs, Wi-Fi is en0, but use the list printed by the program. Start ordinary network activity during capture to help collect packets

Windows System

First we Install Npcap (Scapy needs it to capture packets on Windows). Keep the default installer options. Then, Run these commands in PowerShell in the project folder:

py -m venv .venv

.\.venv\Scripts\Activate.ps1

py -m pip install -r requirements.txt

py test_parser.py

py test_filters.py

py test_display.py

py main.py

If PowerShell blocks Activate.ps1, use Command Prompt and run .venv\Scripts\activate.bat instead. The program lists interfaces by number with their friendly names (for example Wi-Fi) and marks the default one; press Enter to use it. If capture fails with a permission error, open the terminal with Run as administrator

Running the Program now

From the activated environment we should now run:

python main.py (for windows you can write it as py main.py)

After this:

- Select the interface you want to capture on (a numbered interface, or its name, or just press enter to select the default interface)
- Set the capture limits by entering the maximum packet or frame count(or you can put the default value 50) as well as the duration in seconds(you can put the default value as 15). The capture will stop as soon as any limit is reached
- Generate ordinary traffic where you can open a website or run a normal DNS lookup in another terminal during the capture
- Then you can use the menu through options 1-4 as per the user's wish

Menu and it's options meaning here

1. Show all parsed IPv4 packets
2. Filter by protocol, source/destination IP and/or source/destination port. Leave a field blank to skip it. Invalid values (for example a port above 65535 or a malformed IP) are reported and no filter is applied
3. Inspect the full details for a packet number
4. Exit

How a packet is decoded from bytes

All multi-byte fields are big-endian (network byte order) and are read with int.from_bytes(data, "big"). Offsets are counted from the start of each header.

Ethernet header (14 bytes)

Bytes 12–13 are the EtherType. 0x0800 means IPv4; anything else is skipped. For an ordinary frame the IPv4 header starts at byte 14.

802.1Q VLAN tag (optional, 4 bytes)

If the EtherType is 0x8100, a VLAN tag has been inserted: bytes 14–15 hold the tag and bytes 16–17 hold the real EtherType, so the IPv4 header starts at byte 18.

IPv4 header (20–60 bytes)

Bytes	Field	How it is extracted
0	Version, header length	version = byte >> 4; header_length = (byte & 0x0F) * 4 (the IHL counts 4-byte words)
2–3	Total length	IPv4 header + data, excluding Ethernet
6–7	Flags, fragment offset	fragment_offset = value & 0x1FFF; More Fragments flag = value & 0x2000
8	TTL	One byte
9	Protocol	6 = TCP, 17 = UDP, 1 = ICMP — this is how TCP and UDP are identified automatically

10–11	Header checksum	Shown in hexadecimal
12–15 / 16–19	Source / destination IP	4 bytes each, converted to dotted form with ipaddress.IPv4Address

The TCP/UDP header starts right after the IPv4 header, at ip_start + header_length. The IPv4 data ends at ip_start + total_length, so Ethernet padding is never counted as payload.

TCP header (20–60 bytes)

Bytes	Field	How it is extracted
0–1 / 2–3	Source / destination port	16-bit integers
4–7 / 8–11	Sequence / acknowledgment number	32-bit integers
12	Header length (data offset)	(byte >> 4) * 4; values below 20 are invalid
13	Flags	One bit each: FIN 0x01, SYN 0x02, RST 0x04, PSH 0x08, ACK 0x10, URG 0x20, ECE 0x40, CWR 0x80 (e.g. 0x18 = PSH + ACK)
14–15	Window size	16-bit integer
16–17	Checksum	Shown in hexadecimal

TCP payload length = IP total length − IP header length − TCP header length.

UDP header (8 bytes)

Source port at bytes 0–1, destination port at 2–3, length at 4–5 and checksum at 6–7. The UDP length includes the 8-byte header, so payload length = UDP length − 8. A UDP length below 8, or larger than the IP data (except in the first fragment of a fragmented packet), is rejected.

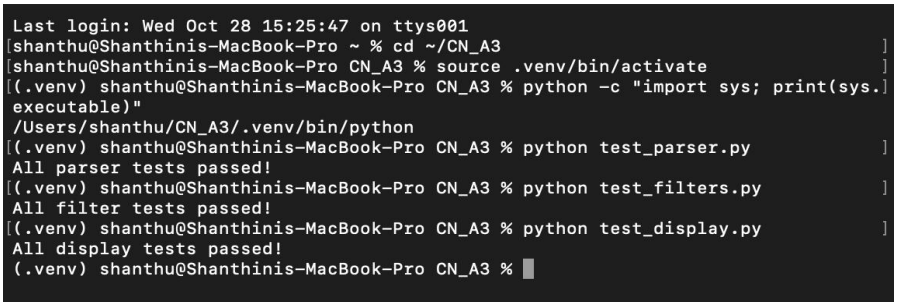
Payload

Shown 16 bytes per line in hexadecimal, with the same bytes as text alongside. Printable ASCII bytes (0x20–0x7E) appear as characters (e.g. 0x48 = H); all other bytes appear as ".", because many payloads are binary or encrypted. A packet with no application data shows a payload length of 0 and "Empty payload."

Testing the system

Here we created and recorded various test cases to check how the system works. Tests were run on two machines: macOS (interface en0, IP 192.168.40.230) and Windows 11 (interface Wi-Fi, IP 192.168.40.117). Each screenshot is from one of these, and packet numbers differ between capture runs.

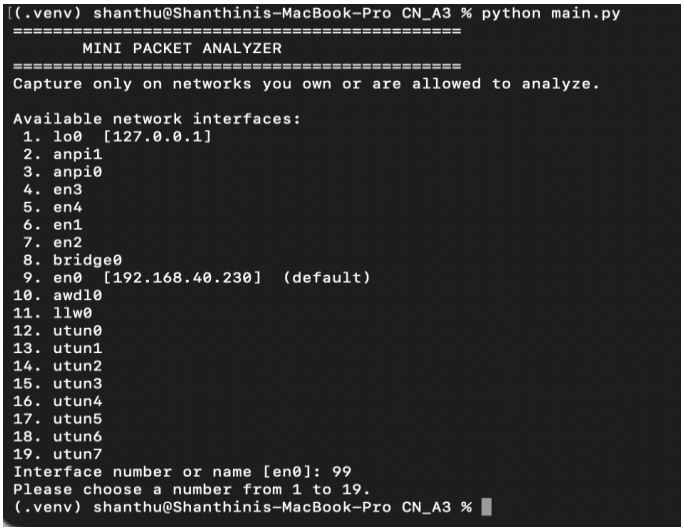
1. **Offline tests: where we check if all three suites passed the virtual environment or not**



Result: All three offline test suites passed inside the virtual environment. These tests build packet bytes in code, so they need no network access or administrator rights.

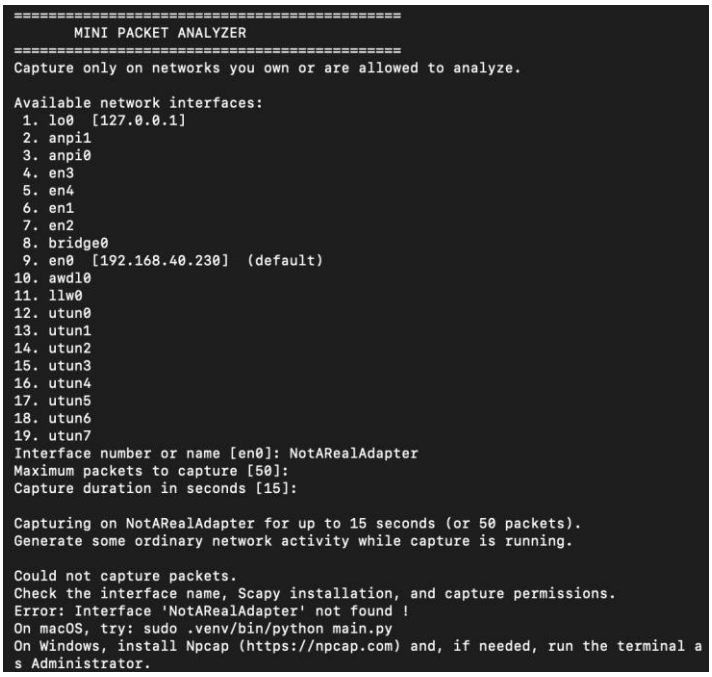
2. **Checking Startup and Interface error tests**

(a) Checking the network interface list and entering an out-of-range interface number



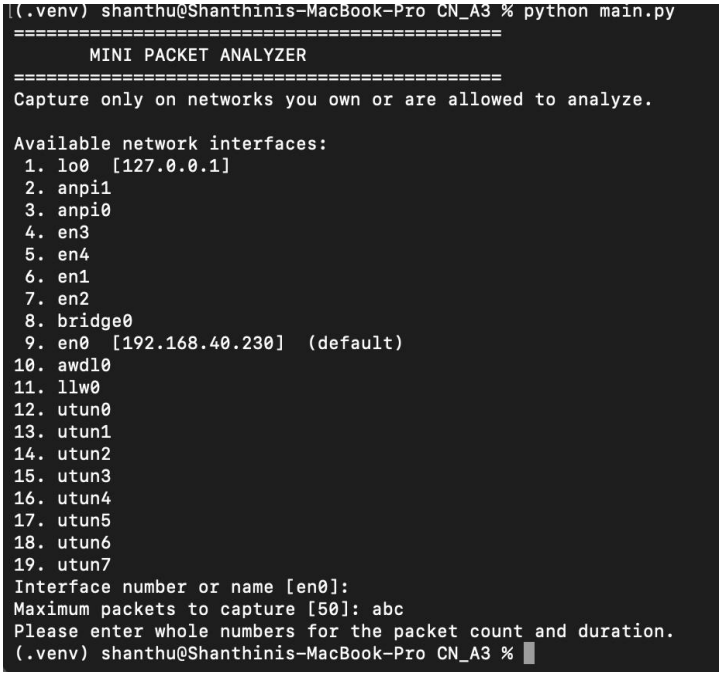
Result: The program listed 19 interfaces by number, showed the IP address next to en0 and marked it as the default. Entering 99 was rejected with "Please choose a number from 1 to 19."

(b) Testing a nonexistent interface name



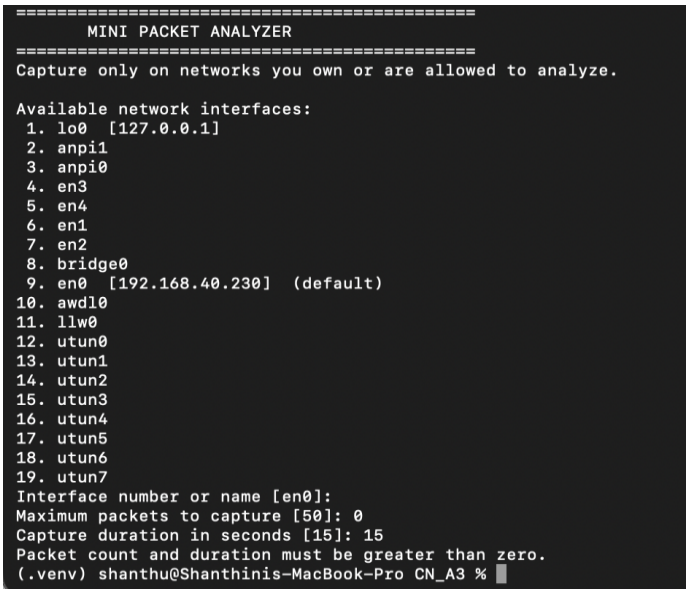
Result: A nonexistent interface name was accepted at the prompt, but capture failed safely with "Could not capture packets", the reason ("Interface 'NotARealAdapter' not found") and setup hints for macOS and Windows, instead of crashing.

(c) Testing a nonnumeric packet count



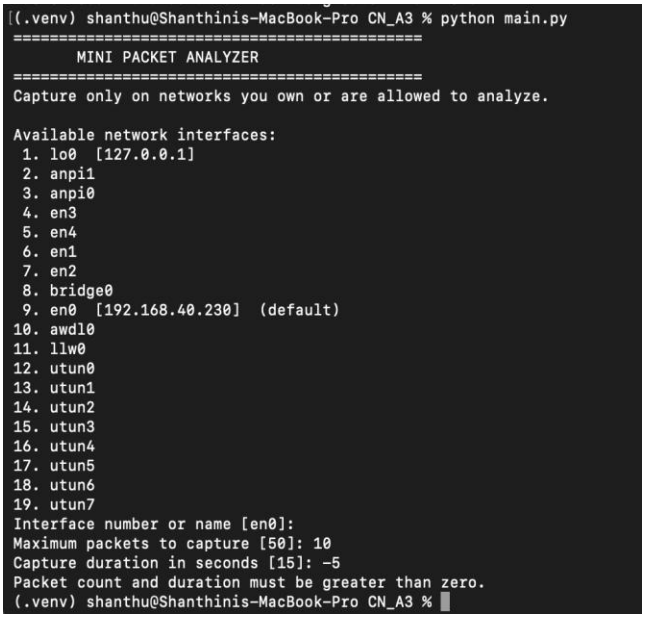
Result: A non-numeric packet count was rejected with "Please enter whole numbers for the packet count and duration."

(d) Testing a packet count of 0



Result: A packet count of 0 was rejected with "Packet count and duration must be greater than zero."

(e) Testing a negative capture duration



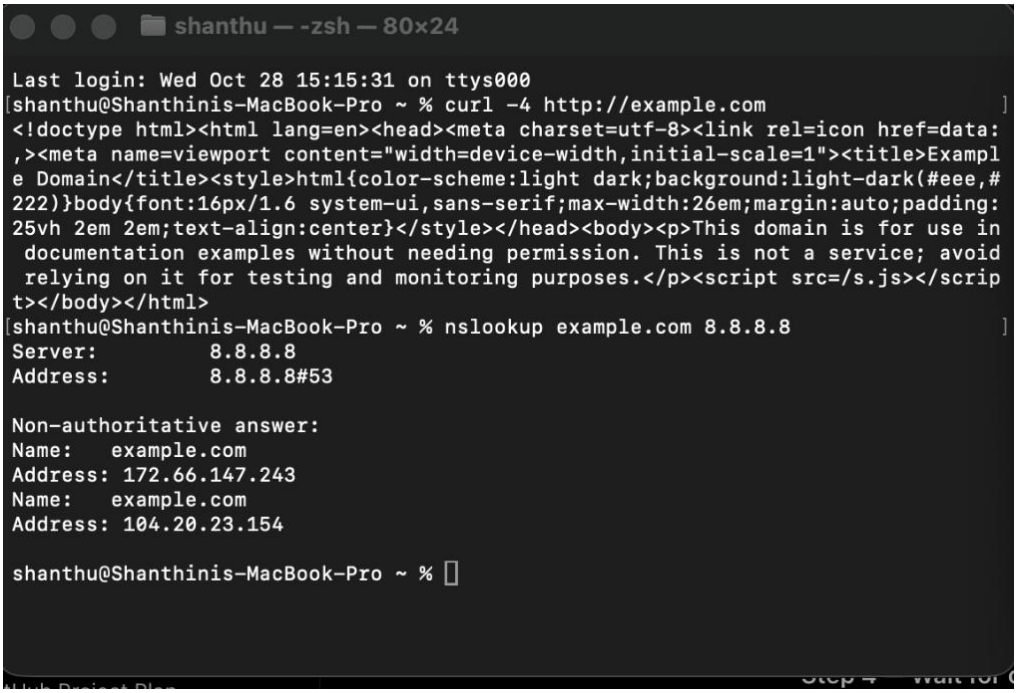
Result: A negative duration (-5) was rejected with the same message.

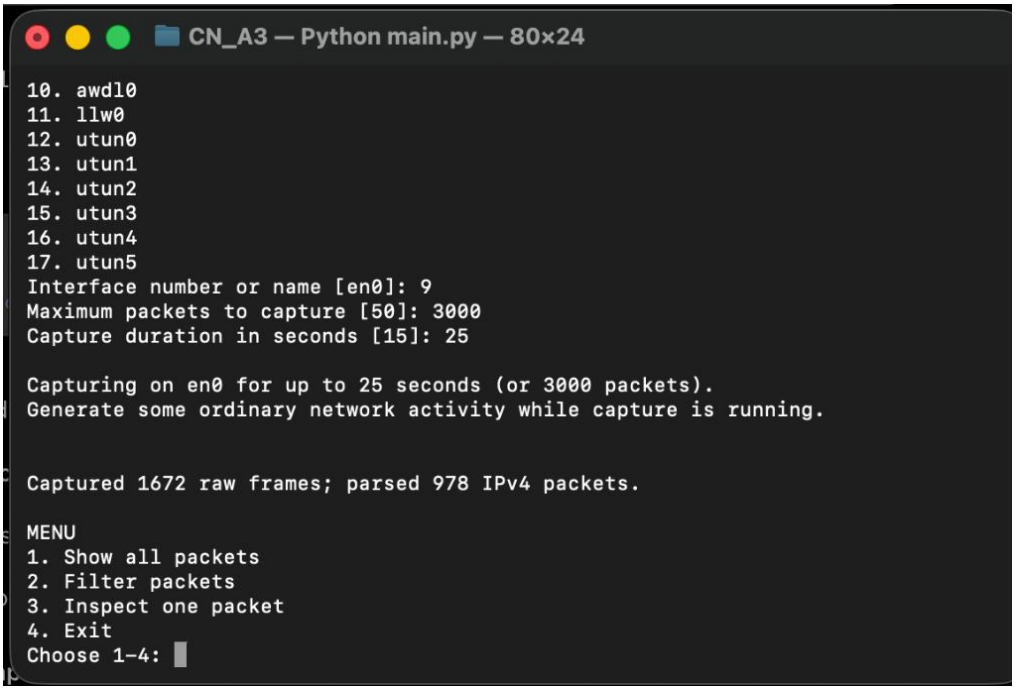
(f) Testing a valid live capture



Result: Selecting interface 9 (en0) with the default limits captured 50 raw frames, of which 15 were IPv4 packets. The other frames (for example ARP and IPv6) are not IPv4, so the parser skips them, as designed.

3. Main Capturing Sessions





Here we ran the packet analyzer on my Mac's Wi-Fi interface (en0) for 25 seconds while generating normal network traffic. Then we tested a HTTP request using curl and performed a DNS lookup using nslookup, which successfully returned IP addresses for example.com. The program captured 1,672 raw frames and parsed 978 IPv4 packets, after which the menu appeared as expected.

4. Summary capturing

```
2. Filter packets
3. Inspect one packet
4. Exit
Choose 1-4: 1

No.   Time          Source IP        Src Port  Destination IP   Dst Port  Protocol  Length
-------------------------------------------------------------------------------------------
1     15:25:39.984  192.168.32.118   50679     239.255.255.250  1900      UDP       212
2     15:25:39.985  192.168.43.243   61659     239.255.255.250  1900      UDP       167
3     15:25:40.089  192.168.55.92    57915     239.255.255.250  3702      UDP       698
4     15:25:40.089  192.168.54.172   57811     239.255.255.250  1900      UDP       167
5     15:25:40.090  192.168.40.151   59745     239.255.255.250  1900      UDP       212
6     15:25:40.090  192.168.40.151   59746     239.255.255.250  1900      UDP       212
7     15:25:40.093  192.168.50.53    5353      224.0.0.251      5353      UDP       470
8     15:25:40.193  192.168.53.52    52592     239.255.255.250  1900      UDP       212
9     15:25:40.214  192.168.40.230   5353      192.168.48.20    5353      UDP       75
10    15:25:40.222  192.168.39.39    56640     239.255.255.250  1900      UDP       212
11    15:25:40.223  192.168.39.39    56891     239.255.255.250  1900      UDP       212
12    15:25:40.250  192.168.39.39    50993     239.255.255.250  3702      UDP       698
13    15:25:40.284  192.168.40.230   51601     192.178.177.113  443       UDP       71
14    15:25:40.289  192.178.177.113  443       192.168.40.230   51601     UDP       68
```

```
968   15:26:04.699  192.168.43.59    56939     239.255.255.250  1900      UDP       179
969   15:26:04.700  192.168.39.10    57229     239.255.255.250  1900      UDP       212
970   15:26:04.764  192.168.40.112   58202     239.255.255.250  1900      UDP       214
971   15:26:04.818  192.168.40.230   58287     35.186.224.35    443       TCP       98
972   15:26:04.828  35.186.224.35    443       192.168.40.230   58287     TCP       66
973   15:26:04.867  192.168.54.98    55283     239.255.255.250  1900      UDP       213
974   15:26:04.867  192.168.48.126   49198     239.255.255.250  1900      UDP       167
975   15:26:04.919  192.168.54.22    52461     239.255.255.250  1900      UDP       212
976   15:26:04.933  192.168.54.22    52463     239.255.255.250  1900      UDP       212
977   15:26:04.953  35.186.224.35    443       192.168.40.230   58287     TCP       94
978   15:26:04.953  192.168.40.230   58287     35.186.224.35    443       TCP       66

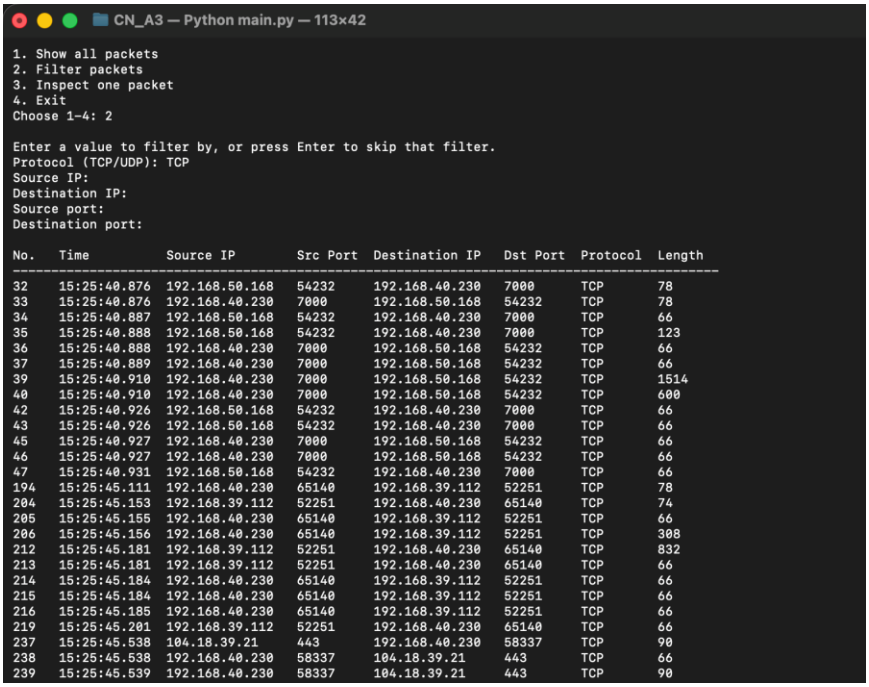
978 packet(s) shown. Length = captured frame size in bytes. '-' = no port (not TCP/UDP, or header incomplete).


MENU
1. Show all packets
2. Filter packets
3. Inspect one packet
4. Exit
Choose 1-4:
```

Selecting option 1 to display the captured packets and checked the summary table. It showed all the required columns, with packet numbers increasing sequentially and timestamps including milliseconds. The bottom of the table confirmed that 978 packets were displayed and explained the meaning of packet length and unavailable ports. Overall, all summary checks passed successfully. The first rows show local discovery traffic (SSDP to 239.255.255.250:1900 and mDNS on port 5353) and the last rows show TCP traffic on port 443.

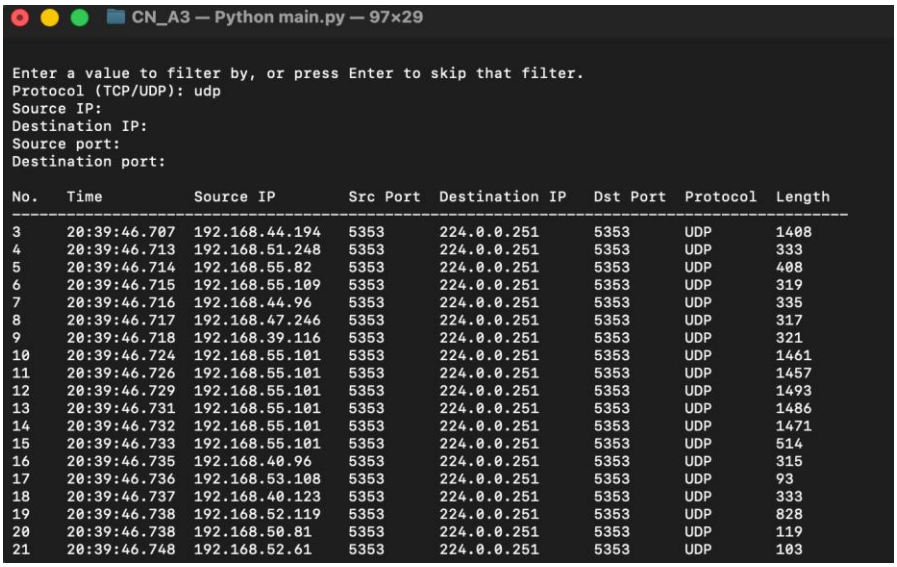
5. Checking all the filters

(a) Filtering by TCP



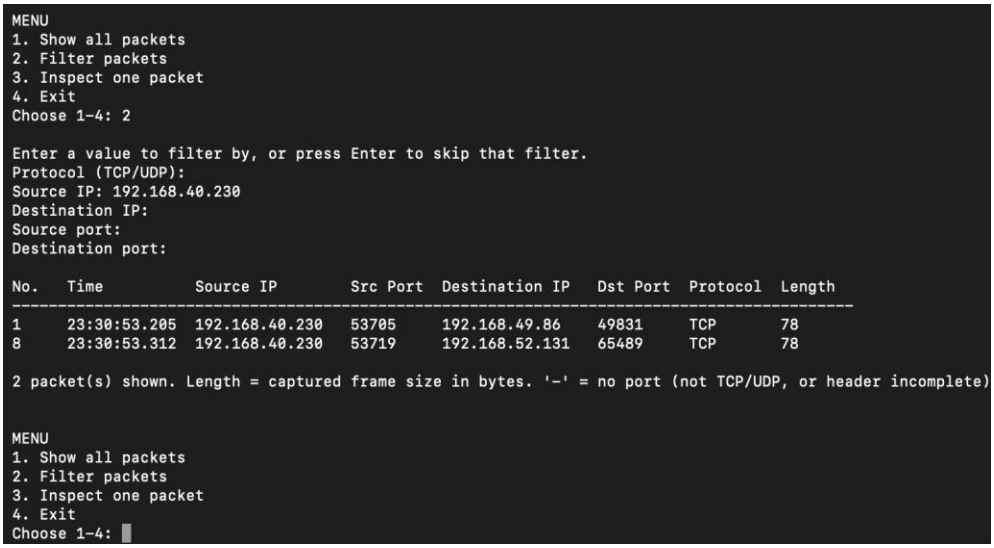
Result: Only TCP rows were returned (for example ports 7000/54232 on the local network and HTTPS on port 443).

(b) Testing the lowercase UDP when typed in the system



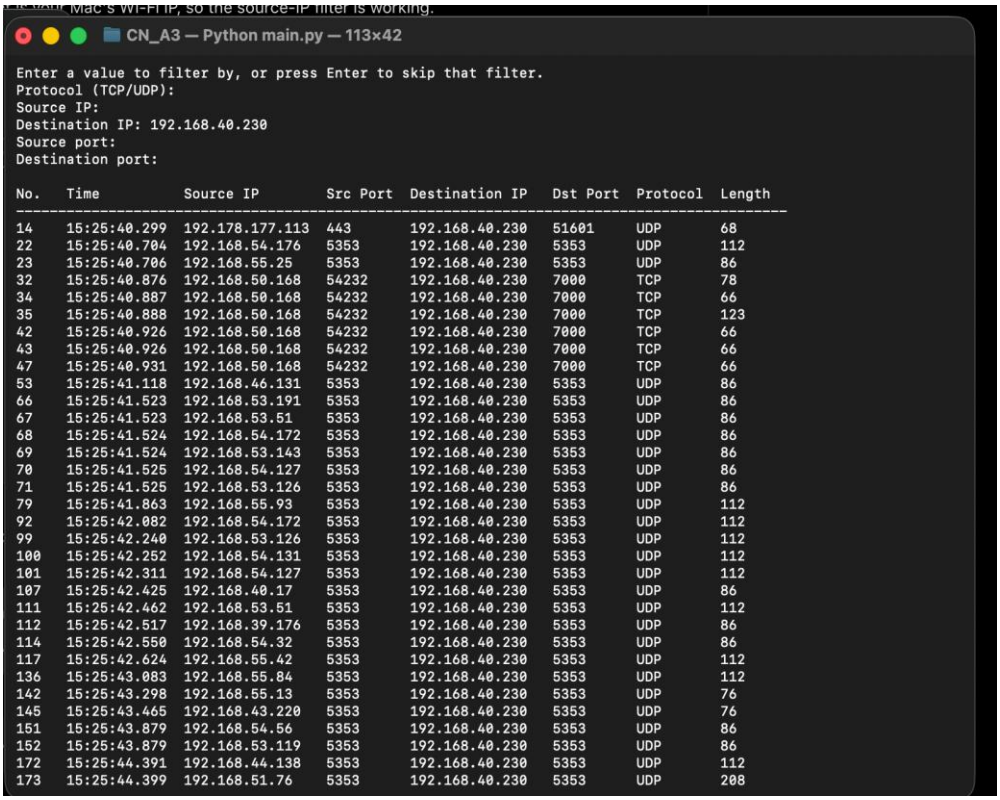
Result: Typing "udp" in lowercase returned only UDP rows, so protocol input is not case sensitive.

(c) Filtering with Mac's Source IP address



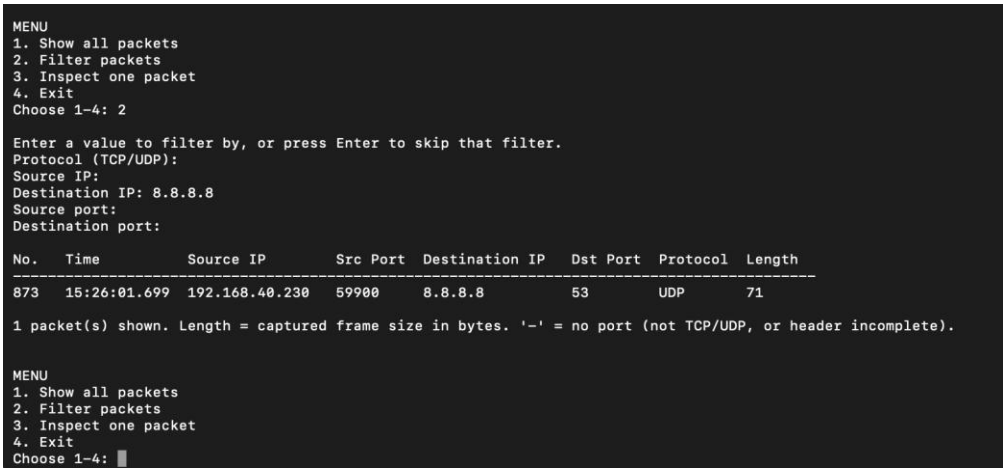
Result: Only packets sent by the Mac (source 192.168.40.230) were returned.

(d) Filtering by destination IP address



Result: Only packets received by the Mac (destination 192.168.40.230) were returned, including UDP replies and incoming TCP segments.

(e) Filtering by destination IP address(8.8.8.8)



Result: Only the DNS query sent to Google's DNS server (UDP, destination port 53) was returned.

(f) Filtering the HTTP requests on the TCP port 80

```
MENU
1. Show all packets
2. Filter packets
3. Inspect one packet
4. Exit
Choose 1-4: 2

Enter a value to filter by, or press Enter to skip that filter.
Protocol (TCP/UDP): TCP
Source IP:
Destination IP:
Source port:
Destination port: 80

No.   Time          Source IP        Src Port  Destination IP   Dst Port  Protocol  Length
-------------------------------------------------------------------------------------------
536   15:25:52.600  192.168.40.230   65141     104.20.23.154    80        TCP       78
538   15:25:52.659  192.168.40.230   65141     104.20.23.154    80        TCP       66
539   15:25:52.660  192.168.40.230   65141     104.20.23.154    80        TCP       140
542   15:25:52.728  192.168.40.230   65141     104.20.23.154    80        TCP       66
552   15:25:53.021  192.168.40.230   65141     104.20.23.154    80        TCP       66
553   15:25:53.021  192.168.40.230   65141     104.20.23.154    80        TCP       66
559   15:25:53.047  192.168.40.230   65141     104.20.23.154    80        TCP       78
562   15:25:53.090  192.168.40.230   65141     104.20.23.154    80        TCP       66

8 packet(s) shown. Length = captured frame size in bytes. '-' = no port (not TCP/UDP, or header incomplete).


MENU
1. Show all packets
2. Filter packets
3. Inspect one packet
4. Exit
Choose 1-4:
```

Result: The client's packets of the HTTP connection to example.com (104.20.23.154) were returned: the connection handshake, the GET request (140 bytes) and the acknowledgements.

(g) Filtering the TCP responses from the source port 80

```
MENU
1. Show all packets
2. Filter packets
3. Inspect one packet
4. Exit
Choose 1-4: 2

Enter a value to filter by, or press Enter to skip that filter.
Protocol (TCP/UDP): TCP
Source IP:
Destination IP:
Source port: 80
Destination port:

No.   Time          Source IP        Src Port  Destination IP   Dst Port  Protocol  Length
-------------------------------------------------------------------------------------------
537   15:25:52.659  104.20.23.154    80        192.168.40.230   65141     TCP       74
540   15:25:52.721  104.20.23.154    80        192.168.40.230   65141     TCP       66
541   15:25:52.728  104.20.23.154    80        192.168.40.230   65141     TCP       997
551   15:25:53.020  104.20.23.154    80        192.168.40.230   65141     TCP       71
557   15:25:53.046  104.20.23.154    80        192.168.40.230   65141     TCP       71
561   15:25:53.089  104.20.23.154    80        192.168.40.230   65141     TCP       66
563   15:25:53.112  104.20.23.154    80        192.168.40.230   65141     TCP       78

7 packet(s) shown. Length = captured frame size in bytes. '-' = no port (not TCP/UDP, or header incomplete).
```

Result: The server's replies in the same connection were returned, including the 997-byte HTTP response.

(h) Filtering the HTTPS traffic (where the TCP destination port here is 443)

```
Enter a value to filter by, or press Enter to skip that filter.
Protocol (TCP/UDP): TCP
Source IP:
Destination IP:
Source port:
Destination port: 443

No.   Time          Source IP        Src Port  Destination IP   Dst Port  Protocol  Length
-------------------------------------------------------------------------------------------
238   15:25:45.538  192.168.40.230   58337     104.18.39.21     443       TCP       66
239   15:25:45.539  192.168.40.230   58337     104.18.39.21     443       TCP       90
240   15:25:45.539  192.168.40.230   58337     104.18.39.21     443       TCP       92
249   15:25:45.894  192.168.40.230   58402     57.144.81.32     443       TCP       78
254   15:25:45.929  192.168.40.230   58402     57.144.81.32     443       TCP       66
255   15:25:45.929  192.168.40.230   58402     57.144.81.32     443       TCP       1446
256   15:25:45.929  192.168.40.230   58402     57.144.81.32     443       TCP       225
262   15:25:45.960  192.168.40.230   58402     57.144.81.32     443       TCP       66
263   15:25:45.960  192.168.40.230   58402     57.144.81.32     443       TCP       66
264   15:25:45.970  192.168.40.230   58402     57.144.81.32     443       TCP       130
265   15:25:45.972  192.168.40.230   58402     57.144.81.32     443       TCP       501
266   15:25:45.973  192.168.40.230   58402     57.144.81.32     443       TCP       690
271   15:25:46.001  192.168.40.230   58402     57.144.81.32     443       TCP       66
272   15:25:46.002  192.168.40.230   58402     57.144.81.32     443       TCP       97
280   15:25:46.097  192.168.40.230   58339     104.18.39.21     443       TCP       66
281   15:25:46.097  192.168.40.230   58339     104.18.39.21     443       TCP       90
282   15:25:46.098  192.168.40.230   58339     104.18.39.21     443       TCP       92
288   15:25:46.203  192.168.40.230   58402     57.144.81.32     443       TCP       66
289   15:25:46.203  192.168.40.230   58402     57.144.81.32     443       TCP       105
290   15:25:46.203  192.168.40.230   58402     57.144.81.32     443       TCP       90
291   15:25:46.203  192.168.40.230   58402     57.144.81.32     443       TCP       66
296   15:25:46.237  192.168.40.230   58402     57.144.81.32     443       TCP       54
297   15:25:46.238  192.168.40.230   58402     57.144.81.32     443       TCP       54
332   15:25:47.109  192.168.40.230   58345     104.18.39.21     443       TCP       66
333   15:25:47.109  192.168.40.230   58345     104.18.39.21     443       TCP       90
334   15:25:47.109  192.168.40.230   58345     104.18.39.21     443       TCP       92
369   15:25:48.460  192.168.40.230   58299     52.22.119.135    443       TCP       66
419   15:25:50.346  192.168.40.230   58337     104.18.39.21     443       TCP       154
420   15:25:50.347  192.168.40.230   58339     104.18.39.21     443       TCP       154
421   15:25:50.347  192.168.40.230   58345     104.18.39.21     443       TCP       154
426   15:25:50.377  192.168.40.230   58339     104.18.39.21     443       TCP       66
427   15:25:50.377  192.168.40.230   58337     104.18.39.21     443       TCP       66
429   15:25:50.405  192.168.40.230   58345     104.18.39.21     443       TCP       66
```

Result: HTTPS traffic from the Mac to several web servers was returned.

(i) Checking the UDP port for 443 using QUIC

```
MENU
1. Show all packets
2. Filter packets
3. Inspect one packet
4. Exit
Choose 1-4: 2

Enter a value to filter by, or press Enter to skip that filter.
Protocol (TCP/UDP): UDP
Source IP:
Destination IP:
Source port:
Destination port: 443

No.   Time          Source IP        Src Port  Destination IP   Dst Port  Protocol  Length
-------------------------------------------------------------------------------------------
13    15:25:40.284  192.168.40.230   51601     192.178.177.113  443       UDP       71
374   15:25:48.607  192.168.40.230   51601     192.178.177.113  443       UDP       77
959   15:26:04.258  192.168.40.230   54545     57.144.49.32     443       UDP       80

3 packet(s) shown. Length = captured frame size in bytes. '-' = no port (not TCP/UDP, or header incomplete).
```

Result: Three QUIC packets (HTTP/3 over UDP) were found.

(j) Testing the destination port 0080

```
Enter a value to filter by, or press Enter to skip that filter.
Protocol (TCP/UDP):
Source IP:
Destination IP:
Source port:
Destination port: 0080

No.   Time          Source IP        Src Port  Destination IP   Dst Port  Protocol  Length
-------------------------------------------------------------------------------------------
536   15:25:52.600  192.168.40.230   65141     104.20.23.154    80        TCP       78
538   15:25:52.659  192.168.40.230   65141     104.20.23.154    80        TCP       66
539   15:25:52.660  192.168.40.230   65141     104.20.23.154    80        TCP       140
542   15:25:52.728  192.168.40.230   65141     104.20.23.154    80        TCP       66
552   15:25:53.021  192.168.40.230   65141     104.20.23.154    80        TCP       66
553   15:25:53.021  192.168.40.230   65141     104.20.23.154    80        TCP       66
559   15:25:53.047  192.168.40.230   65141     104.20.23.154    80        TCP       78
562   15:25:53.090  192.168.40.230   65141     104.20.23.154    80        TCP       66

8 packet(s) shown. Length = captured frame size in bytes. '-' = no port (not TCP/UDP, or header incomplete).
```

Result: The same 8 packets as (f) were returned, because ports are compared as numbers, so leading zeros do not matter.

(k) Combined UDP with destination IP and port

```
MENU
1. Show all packets
2. Filter packets
3. Inspect one packet
4. Exit
Choose 1-4: 2

Enter a value to filter by, or press Enter to skip that filter.
Protocol (TCP/UDP): UDP
Source IP:
Destination IP: 8.8.8.8
Source port:
Destination port: 53

No.   Time          Source IP        Src Port  Destination IP   Dst Port  Protocol  Length
-------------------------------------------------------------------------------------------
873   15:26:01.699  192.168.40.230   59900     8.8.8.8          53        UDP       71

1 packet(s) shown. Length = captured frame size in bytes. '-' = no port (not TCP/UDP, or header incomplete).


MENU
1. Show all packets
2. Filter packets
3. Inspect one packet
4. Exit
Choose 1-4:
```

Result: All three conditions must match, so only the DNS query was returned.

(l) Combined TCP with source IP and destination port 80

```
MENU
1. Show all packets
2. Filter packets
3. Inspect one packet
4. Exit
Choose 1-4: 2

Enter a value to filter by, or press Enter to skip that filter.
Protocol (TCP/UDP): TCP
Source IP: 192.168.40.230
Destination IP:
Source port:
Destination port: 80

No.   Time          Source IP        Src Port  Destination IP   Dst Port  Protocol  Length
-------------------------------------------------------------------------------------------
536   15:25:52.600  192.168.40.230   65141     104.20.23.154    80        TCP       78
538   15:25:52.659  192.168.40.230   65141     104.20.23.154    80        TCP       66
539   15:25:52.660  192.168.40.230   65141     104.20.23.154    80        TCP       140
542   15:25:52.728  192.168.40.230   65141     104.20.23.154    80        TCP       66
552   15:25:53.021  192.168.40.230   65141     104.20.23.154    80        TCP       66
553   15:25:53.021  192.168.40.230   65141     104.20.23.154    80        TCP       66
559   15:25:53.047  192.168.40.230   65141     104.20.23.154    80        TCP       78
562   15:25:53.090  192.168.40.230   65141     104.20.23.154    80        TCP       66

8 packet(s) shown. Length = captured frame size in bytes. '-' = no port (not TCP/UDP, or header incomplete).


MENU
1. Show all packets
2. Filter packets
3. Inspect one packet
4. Exit
Choose 1-4:
```

Result: The same 8 packets as (f) were returned, confirming that combined filters narrow the results correctly.

(m) Testing the destination port filter with no matching packets (port 12345)

```
MENU
1. Show all packets
2. Filter packets
3. Inspect one packet
4. Exit
Choose 1-4: 2

Enter a value to filter by, or press Enter to skip that filter.
Protocol (TCP/UDP):
Source IP:
Destination IP:
Source port:
Destination port: 12345
No matching IPv4 packets to display.
```

Result: No packet used this port, so "No matching IPv4 packets to display." was shown.

(n) Testing the combined TCP protocol and destination port 53 filter

```
MENU
1. Show all packets
2. Filter packets
3. Inspect one packet
4. Exit
Choose 1-4: 2

Enter a value to filter by, or press Enter to skip that filter.
Protocol (TCP/UDP): TCP
Source IP:
Destination IP:
Source port:
Destination port: 53
No matching IPv4 packets to display.
```

Result: No match, because the DNS traffic in this capture used UDP, not TCP.

(o) Testing the filter option with all fields left blank

```
MENU
1. Show all packets
2. Filter packets
3. Inspect one packet
4. Exit
Choose 1-4: 2

Enter a value to filter by, or press Enter to skip that filter.
Protocol (TCP/UDP):
Source IP:
Destination IP:
Source port:
Destination port:

No.   Time          Source IP        Src Port  Destination IP   Dst Port  Protocol  Length
-------------------------------------------------------------------------------------------
1     04:27:44.452  192.168.40.147   56780     239.255.255.250  1900      UDP       212
2     04:27:44.452  192.168.40.147   56783     239.255.255.250  1900      UDP       212
3     04:27:44.458  192.168.52.248   64608     239.255.255.250  1900      UDP       136
4     04:27:44.458  192.168.52.248   64608     239.255.255.250  1900      UDP       179
5     04:27:44.554  192.168.43.21    5353      224.0.0.251      5353      UDP       366
6     04:27:44.562  192.168.43.21    5353      224.0.0.251      5353      UDP       304
7     04:27:44.574  192.168.50.141   5353      224.0.0.251      5353      UDP       858
8     04:27:44.578  192.168.54.69    57782     239.255.255.250  1900      UDP       212
9     04:27:44.581  192.168.39.252   5353      224.0.0.251      5353      UDP       490
10    04:27:44.651  192.168.55.142   49867     239.255.255.250  1900      UDP       167

10 packet(s) shown. Length = captured frame size in bytes. '-' = no port (not TCP/UDP, or header incomplete).
```

Result: With no filters, every captured packet was shown (10 packets in this capture).

(p) Testing invalid protocol input (ICMP)

```
MENU
1. Show all packets
2. Filter packets
3. Inspect one packet
4. Exit
Choose 1-4: 2

Enter a value to filter by, or press Enter to skip that filter.
Protocol (TCP/UDP): ICMP
Source IP:
Destination IP:
Source port:
Destination port:
Invalid filter: Protocol must be TCP, UDP, or blank. No filter applied.
```

Result: Rejected, because protocol filtering supports only TCP and UDP, as the assignment specifies.

(q) Testing invalid source IP address

```
MENU
1. Show all packets
2. Filter packets
3. Inspect one packet
4. Exit
Choose 1-4: 2

Enter a value to filter by, or press Enter to skip that filter.
Protocol (TCP/UDP):
Source IP: 192.168.1.300
Destination IP:
Source port:
Destination port:
Invalid filter: Source IP must be an IPv4 address such as 192.168.1.10. No filter applied.
```

Result: Rejected, because 300 is not a valid octet (each part of an IPv4 address must be 0–255).

(r) Testing invalid destination IP address

```
MENU
1. Show all packets
2. Filter packets
3. Inspect one packet
4. Exit
Choose 1-4: 2

Enter a value to filter by, or press Enter to skip that filter.
Protocol (TCP/UDP):
Source IP:
Destination IP: hello
Source port:
Destination port:
Invalid filter: Destination IP must be an IPv4 address such as 192.168.1.10. No filter applied.
```

Result: Rejected as not an IPv4 address.

(s) Testing numeric input in the Source IP field

```
MENU
1. Show all packets
2. Filter packets
3. Inspect one packet
4. Exit
Choose 1-4: 2

Enter a value to filter by, or press Enter to skip that filter.
Protocol (TCP/UDP):
Source IP: 5353
Destination IP:
Source port:
Destination port:
Invalid filter: Source IP must be an IPv4 address such as 192.168.1.10. No filter applied.
```

Result: Rejected; this catches the mistake of typing a port number into an IP field.

(t) Testing an invalid source port value

```
MENU
1. Show all packets
2. Filter packets
3. Inspect one packet
4. Exit
Choose 1-4: 2

Enter a value to filter by, or press Enter to skip that filter.
Protocol (TCP/UDP):
Source IP:
Destination IP:
Source port: 99999
Destination port:
Invalid filter: Source port must be a whole number from 0 to 65535. No filter applied.
```

Result: Rejected, because ports are 16-bit numbers (0–65535).

(u) Testing a non-numeric destination port value

```
MENU
1. Show all packets
2. Filter packets
3. Inspect one packet
4. Exit
Choose 1-4: 2

Enter a value to filter by, or press Enter to skip that filter.
Protocol (TCP/UDP):
Source IP:
Destination IP:
Source port:
Destination port: abc
Invalid filter: Destination port must be a whole number from 0 to 65535. No filter applied.
```

Result: Rejected as not a number.

(v) Testing a negative destination port value

```
MENU
1. Show all packets
2. Filter packets
3. Inspect one packet
4. Exit
Choose 1-4: 2

Enter a value to filter by, or press Enter to skip that filter.
Protocol (TCP/UDP):
Source IP:
Destination IP:
Source port:
Destination port: -1
Invalid filter: Destination port must be a whole number from 0 to 65535. No filter applied.
```

Result: Rejected as outside 0–65535.

6. Detailed Packet Inspection

(a) Inspecting an HTTP request packet

```
Enter a value to filter by, or press Enter to skip that filter.
Protocol (TCP/UDP): TCP
Source IP:
Destination IP:
Source port:
Destination port: 80

No.   Time          Source IP        Src Port  Destination IP   Dst Port  Protocol  Length
-------------------------------------------------------------------------------------------
1301  04:55:41.398  192.168.40.117   59827     172.66.147.243   80        TCP       66
1303  04:55:41.399  192.168.40.117   59827     172.66.147.243   80        TCP       54
1304  04:55:41.400  192.168.40.117   59827     172.66.147.243   80        TCP       129
1336  04:55:41.794  192.168.40.117   59827     172.66.147.243   80        TCP       54
1340  04:55:41.972  192.168.40.117   59827     172.66.147.243   80        TCP       54
1345  04:55:42.022  192.168.40.117   59827     172.66.147.243   80        TCP       66
1347  04:55:42.024  192.168.40.117   59827     172.66.147.243   80        TCP       54

7 packet(s) shown. Length = captured frame size in bytes. '-' = no port (not TCP/UDP, or header incomplete).
```

```
MENU
1. Show all packets
2. Filter packets
3. Inspect one packet
4. Exit
Choose 1-4: 3
Enter the packet number to inspect: 1304

================================================
PACKET #1304
Time: 04:55:41.400
Packet length: 129 bytes (captured frame)

IP HEADER
------------------------------
Version: 4
Header length: 20 bytes
Total length: 115 bytes
TTL: 128
Protocol: TCP (number 6)
Header checksum: 0x532A
Source IP: 192.168.40.117
Destination IP: 172.66.147.243
Fragment offset: 0

TCP HEADER
------------------------------
Source port: 59827
Destination port: 80
Sequence number: 2219852521
Acknowledgment number: 3297138727
Header length: 20 bytes
Flags: PSH, ACK
Window size: 255
Checksum: 0x186E
Payload length: 75 bytes

PAYLOAD
------------------------------
Payload length: 75 bytes
Offset  Hex                                             Text
0000    47 45 54 20 2F 20 48 54 54 50 2F 31 2E 31 0D 0A  GET / HTTP/1.1..
0010    48 6F 73 74 3A 20 65 78 61 6D 70 6C 65 2E 63 6F  Host: example.co
0020    6D 0D 0A 55 73 65 72 2D 41 67 65 6E 74 3A 20 63  m..User-Agent: c
0030    75 72 6C 2F 38 2E 32 31 2E 30 0D 0A 41 63 63 65  url/8.21.0..Acce
0040    70 74 3A 20 2A 2F 2A 0D 0A 0D 0A                 pt: */*....
================================================
```

Result: This is the GET request sent by curl to example.com (packet 1304). The IP header shows version 4, a 20-byte header, total length 115 and protocol TCP (6). The TCP header shows destination port 80 and flags PSH, ACK. The payload text shows "GET / HTTP/1.1", "Host: example.com" and "User-Agent: curl/8.21.0". The lengths agree: 20 (IP header) + 20 (TCP header) + 75 (payload) = 115 (IP total length), and 14 (Ethernet header) + 115 = 129 (frame length).

(b) Inspecting an HTTP response packet

```
MENU
1. Show all packets
2. Filter packets
3. Inspect one packet
4. Exit
Choose 1-4: 2

Enter a value to filter by, or press Enter to skip that filter.
Protocol (TCP/UDP): TCP
Source IP:
Destination IP:
Source port: 80
Destination port:

No.   Time          Source IP        Src Port  Destination IP   Dst Port  Protocol  Length
-----------------------------------------------------------------------------------------
1302  04:55:41.398  172.66.147.243   80        192.168.40.117   59827     TCP       66
1305  04:55:41.400  172.66.147.243   80        192.168.40.117   59827     TCP       54
1318  04:55:41.671  172.66.147.243   80        192.168.40.117   59827     TCP       54
1319  04:55:41.675  172.66.147.243   80        192.168.40.117   59827     TCP       985
1339  04:55:41.970  172.66.147.243   80        192.168.40.117   59827     TCP       59
1344  04:55:42.022  172.66.147.243   80        192.168.40.117   59827     TCP       59
1346  04:55:42.023  172.66.147.243   80        192.168.40.117   59827     TCP       54

7 packet(s) shown. Length = captured frame size in bytes. '-' = no port (not TCP/UDP, or header incomplete).
```

```
4. Exit
Choose 1-4: 3
Enter the packet number to inspect: 1319

================================================
PACKET #1319
Time: 04:55:41.675
Packet length: 985 bytes (captured frame)

IP HEADER
------------------------------
Version: 4
Header length: 20 bytes
Total length: 971 bytes
TTL: 51
Protocol: TCP (number 6)
Header checksum: 0xCD3D
Source IP: 172.66.147.243
Destination IP: 192.168.40.117
Fragment offset: 0

TCP HEADER
------------------------------
Source port: 80
Destination port: 59827
Sequence number: 3297138727
Acknowledgment number: 2219852596
Header length: 20 bytes
Flags: PSH, ACK
Window size: 16
Checksum: 0xD23A
Payload length: 931 bytes

PAYLOAD
------------------------------
Payload length: 931 bytes
Offset  Hex                                                     Text
0000    48 54 54 50 2F 31 2E 31 20 32 30 30 20 4F 4B 0D  HTTP/1.1 200 OK.
0010    0A 44 61 74 65 3A 20 54 68 75 2C 20 32 39 20 4F  .Date: Thu, 29 O
0020    63 74 20 32 30 32 36 20 32 33 3A 32 35 3A 34 32  ct 2026 23:25:42
0030    20 47 4D 54 0D 0A 43 6F 6E 74 65 6E 74 2D 54 79   GMT..Content-Ty
0040    70 65 3A 20 74 65 78 74 2F 68 74 6D 6C 3B 20 63  pe: text/html; c
0050    68 61 72 73 65 74 3D 75 74 66 2D 38 0D 0A 54 72  harset=utf-8..Tr
0060    61 6E 73 66 65 72 2D 45 6E 63 6F 64 69 6E 67 3A  ansfer-Encoding:
0070    20 63 68 75 6E 6B 65 64 0D 0A 43 6F 6E 6E 65 63   chunked..Connec
0080    74 69 6F 6E 3A 20 6B 65 65 70 2D 61 6C 69 76 65  tion: keep-alive
0090    0D 0A 53 65 72 76 65 72 3A 20 63 6C 6F 75 64 66  ..Server: cloudf
00A0    6C 61 72 65 0D 0A 4C 61 73 74 2D 4D 6F 64 69 66  lare..Last-Modif
00B0    69 65 64 3A 20 53 61 74 2C 20 32 34 20 4F 63 74  ied: Sat, 24 Oct
00C0    20 32 30 32 36 20 31 31 3A 33 34 3A 34 30 20 47   2026 11:34:40 G
00D0    4D 54 0D 0A 41 6C 6C 6F 77 3A 20 47 45 54 2C 20  MT..Allow: GET,
00E0    48 45 41 44 0D 0A 41 63 63 65 70 74 2D 52 61 6E  HEAD..Accept-Ran
00F0    67 65 73 3A 20 62 79 74 65 73 0D 0A 41 67 65 3A  ges: bytes..Age:
0100    20 32 35 35 38 0D 0A 63 66 2D 63 61 63 68 65 2D   2558..cf-cache-
0110    73 74 61 74 75 73 3A 20 48 49 54 0D 0A 43 46 2D  status: HIT..CF-
0120    52 41 59 3A 20 61 35 32 35 66 30 65 33 38 62 35  RAY: a525f0e38b5
0130    31 35 39 66 34 2D 44 45 4C 0D 0A 61 6C 74 2D 73  159f4-DEL..alt-s
0140    76 63 3A 20 68 33 3D 22 3A 34 34 33 22 3B 20 6D  vc: h3=":443"; m
0150    61 3D 38 36 34 30 30 0D 0A 0D 0A 32 34 31 0D 0A  a=86400....241..
0160    3C 21 64 6F 63 74 79 70 65 20 68 74 6D 6C 3E 3C  <!doctype html><
0170    68 74 6D 6C 20 6C 61 6E 67 3D 65 6E 3E 3C 68 65  html lang=en><he
0180    61 64 3E 3C 6D 65 74 61 20 63 68 61 72 73 65 74  ad><meta charset
0190    3D 75 74 66 2D 38 3E 3C 6C 69 6E 6B 20 72 65 6C  =utf-8><link rel
01A0    3D 69 63 6F 6E 20 68 72 65 66 3D 64 61 74 61 3A  =icon href=data:
```

Result: The server's reply (packet 1319) comes from source port 80. The payload begins "HTTP/1.1 200 OK" followed by the response headers and HTML. The payload length is 971 − 20 − 20 = 931 bytes. Its acknowledgment number (2219852596) equals the request's sequence number (2219852521) plus the request's 75 payload bytes, which shows the server received the whole request.

(c) Inspecting a TCP SYN packet

```
MENU
1. Show all packets
2. Filter packets
3. Inspect one packet
4. Exit
Choose 1-4: 3
Enter the packet number to inspect: 1301

================================================
PACKET #1301
Time: 04:55:41.398
Packet length: 66 bytes (captured frame)

IP HEADER
----------------------------
Version: 4
Header length: 20 bytes
Total length: 52 bytes
TTL: 128
Protocol: TCP (number 6)
Header checksum: 0x536B
Source IP: 192.168.40.117
Destination IP: 172.66.147.243
Fragment offset: 0

TCP HEADER
----------------------------
Source port: 59827
Destination port: 80
Sequence number: 2219852520
Acknowledgment number: 0
Header length: 32 bytes
Flags: SYN
Window size: 65535
Checksum: 0x9480
Payload length: 0 bytes

PAYLOAD
----------------------------
Payload length: 0 bytes
Empty payload.
================================================
```

Result: The first packet of the connection (packet 1301) has only the SYN flag, an acknowledgment number of 0 and an empty payload ("Payload length: 0 bytes", "Empty payload."). The TCP header is 32 bytes rather than 20 because a SYN carries TCP options.

(d) Inspecting a TCP SYN-ACK packet

```
MENU
1. Show all packets
2. Filter packets
3. Inspect one packet
4. Exit
Choose 1-4: 3
Enter the packet number to inspect: 1302

================================================
PACKET #1302
Time: 04:55:41.398
Packet length: 66 bytes (captured frame)

IP HEADER
----------------------------
Version: 4
Header length: 20 bytes
Total length: 52 bytes
TTL: 51
Protocol: TCP (number 6)
Header checksum: 0x1E71
Source IP: 172.66.147.243
Destination IP: 192.168.40.117
Fragment offset: 0

TCP HEADER
----------------------------
Source port: 80
Destination port: 59827
Sequence number: 3297138726
Acknowledgment number: 2219852521
Header length: 32 bytes
Flags: SYN, ACK
Window size: 65535
Checksum: 0x77F9
Payload length: 0 bytes

PAYLOAD
----------------------------
Payload length: 0 bytes
Empty payload.
================================================
```

Result: The server answers with SYN, ACK (packet 1302). Its acknowledgment number (2219852521) is the SYN's sequence number (2219852520) + 1, as the TCP handshake requires.

(e) Inspecting a DNS query packet

```
MENU
1. Show all packets
2. Filter packets
3. Inspect one packet
4. Exit
Choose 1-4: 2

Enter a value to filter by, or press Enter to skip that filter.
Protocol (TCP/UDP): UDP
Source IP:
Destination IP: 8.8.8.8
Source port:
Destination port: 53

No.   Time          Source IP        Src Port  Destination IP   Dst Port  Protocol  Length
-------------------------------------------------------------------------------------------
709   05:05:53.221  192.168.40.117   53583     8.8.8.8          53        UDP       80
711   05:05:53.230  192.168.40.117   53584     8.8.8.8          53        UDP       84
714   05:05:53.272  192.168.40.117   53585     8.8.8.8          53        UDP       84
716   05:05:53.290  192.168.40.117   53586     8.8.8.8          53        UDP       78
718   05:05:53.301  192.168.40.117   53587     8.8.8.8          53        UDP       78
721   05:05:53.341  192.168.40.117   53588     8.8.8.8          53        UDP       71
723   05:05:53.349  192.168.40.117   53589     8.8.8.8          53        UDP       71

7 packet(s) shown. Length = captured frame size in bytes. '-' = no port (not TCP/UDP, or header incomplete).
```

```
MENU
1. Show all packets
2. Filter packets
3. Inspect one packet
4. Exit
Choose 1-4: 3
Enter the packet number to inspect: 711

================================================
PACKET #711
Time: 05:05:53.230
Packet length: 84 bytes (captured frame)

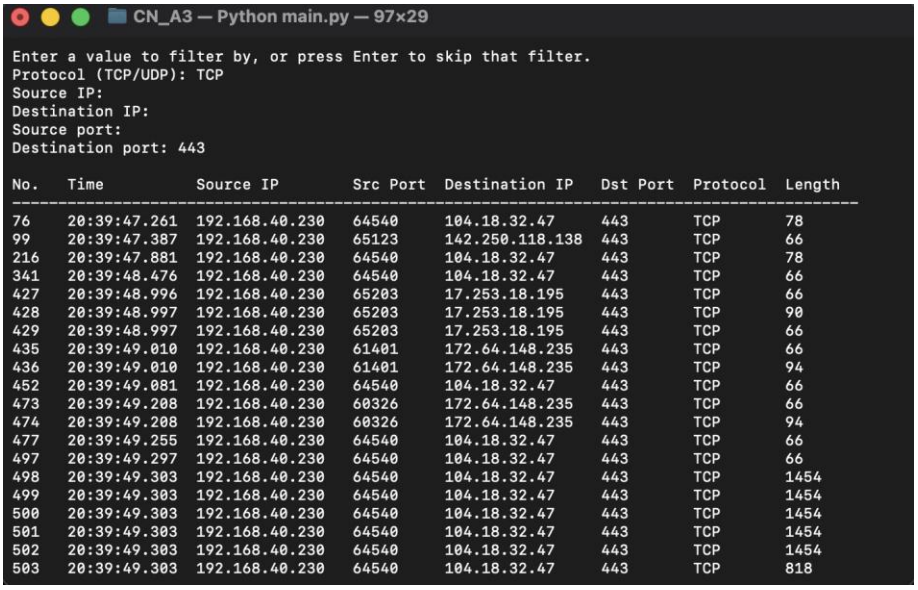
IP HEADER
------------------------------
Version: 4
Header length: 20 bytes
Total length: 70 bytes
TTL: 128
Protocol: UDP (number 17)
Header checksum: 0x969C
Source IP: 192.168.40.117
Destination IP: 8.8.8.8
Fragment offset: 0

UDP HEADER
------------------------------
Source port: 53584
Destination port: 53
Length: 50 bytes
Checksum: 0xABEA
Payload length: 42 bytes

PAYLOAD
------------------------------
Payload length: 42 bytes
Offset  Hex                                             Text
0000    00 02 01 00 00 01 00 00 00 00 00 00 07 65 78 61  .............exa
0010    6D 70 6C 65 03 63 6F 6D 05 69 69 69 74 64 03 65  mple.com.iiitd.e
0020    64 75 02 69 6E 00 00 01 00 01                    du.in.....
================================================
```

Result: A UDP packet to 8.8.8.8 port 53 (packet 711). UDP length 50 = 8 (header) + 42 (payload), and IP total length 70 = 20 + 50. In the payload, each part of the name is preceded by its length byte: 07 then "example", 03 then "com". The name appears as "example.com.iiitd.edu.in" because Windows adds its DNS search suffix (iiitd.edu.in) to some queries.

(f) Inspecting an HTTPS packet



```
CN_A3 — Python main.py — 97×29

Enter a value to filter by, or press Enter to skip that filter.
Protocol (TCP/UDP): TCP
Source IP:
Destination IP:
Source port:
Destination port: 443

No.   Time          Source IP        Src Port  Destination IP   Dst Port  Protocol  Length
-------------------------------------------------------------------------------------------
76    20:39:47.261  192.168.40.230   64540     104.18.32.47     443       TCP       78
99    20:39:47.387  192.168.40.230   65123     142.250.118.138  443       TCP       66
216   20:39:47.881  192.168.40.230   64540     104.18.32.47     443       TCP       78
341   20:39:48.476  192.168.40.230   64540     104.18.32.47     443       TCP       66
427   20:39:48.996  192.168.40.230   65203     17.253.18.195    443       TCP       66
428   20:39:48.997  192.168.40.230   65203     17.253.18.195    443       TCP       90
429   20:39:48.997  192.168.40.230   65203     17.253.18.195    443       TCP       66
435   20:39:49.010  192.168.40.230   61401     172.64.148.235   443       TCP       66
436   20:39:49.010  192.168.40.230   61401     172.64.148.235   443       TCP       94
452   20:39:49.081  192.168.40.230   64540     104.18.32.47     443       TCP       66
473   20:39:49.208  192.168.40.230   60326     172.64.148.235   443       TCP       66
474   20:39:49.208  192.168.40.230   60326     172.64.148.235   443       TCP       94
477   20:39:49.255  192.168.40.230   64540     104.18.32.47     443       TCP       66
497   20:39:49.297  192.168.40.230   64540     104.18.32.47     443       TCP       66
498   20:39:49.303  192.168.40.230   64540     104.18.32.47     443       TCP       1454
499   20:39:49.303  192.168.40.230   64540     104.18.32.47     443       TCP       1454
500   20:39:49.303  192.168.40.230   64540     104.18.32.47     443       TCP       1454
501   20:39:49.303  192.168.40.230   64540     104.18.32.47     443       TCP       1454
502   20:39:49.303  192.168.40.230   64540     104.18.32.47     443       TCP       1454
503   20:39:49.303  192.168.40.230   64540     104.18.32.47     443       TCP       818
```

```
PACKET #498
Time: 20:39:49.303
Packet length: 1454 bytes (captured frame)

IP HEADER
------------------------------
Version: 4
Header length: 20 bytes
Total length: 1440 bytes
TTL: 64
Protocol: TCP (number 6)
Header checksum: 0xC388
Source IP: 192.168.40.230
Destination IP: 104.18.32.47
Fragment offset: 0

TCP HEADER
------------------------------
Source port: 64540
Destination port: 443
Sequence number: 851735152
Acknowledgment number: 2820569379
Header length: 32 bytes
Flags: ACK
Window size: 14540
Checksum: 0x18A8
Payload length: 1388 bytes

PAYLOAD
------------------------------
Payload length: 1388 bytes
Offset  Hex                                              Text
0000    17 03 03 1E 07 24 DC 6A BC 99 FE 60 98 4A 9C 52  .....$.j...`.J.R
0010    0A FC 24 24 57 72 12 6A 63 EC A9 FA DB 2E EB B8  ..$$Wr.jc.......
0020    22 81 58 01 F1 AE 7A B4 B9 8C 2D 2E 60 63 74 13  ".X...z...-.`ct.
0030    5A E2 B1 F5 59 C6 50 ED EA 4B 1A 1C 9F 5A D7 BC  Z...Y.P..K...Z..
0040    FC 51 40 DF 54 53 D9 F0 70 90 C9 79 D4 9A D4 B8  .Q@.TS..p..y....
0050    CE 68 6A 2C 22 21 9C 2D 53 38 AB 6A 02 DD F2 D2  .hj,"!.-S8.j....
0060    54 B6 D5 36 70 23 74 33 ED 5F FB 4B 63 47 AE C8  T..6p#t3._.KcG..
0070    EA 65 F5 39 F9 C4 02 BD 02 51 17 B2 31 A2 D4 03  .e.9.....Q..1...
0080    E6 A5 48 9E 74 01 F4 F5 16 02 97 78 EC 88 37 F0  ..H.t......x..7.
0090    CA BB 7B E4 B3 7E 37 CE 83 F5 3F FD 86 CF C7 8F  ..{..~7...?.....
00A0    9A E3 81 6E 50 97 DB AB D8 57 74 8C DB 60 23 6D  ...nP....Wt..`#m
00B0    E3 DA 37 4E 36 62 C4 B5 C7 55 8F E5 AF 6F F9 D5  ..7N6b...U...o..
00C0    F2 6B 8F 69 19 83 FB 1F DD 1E 0D D2 0F 93 A3 C6  .k.i............
00D0    32 36 8F 64 1D 7B 1E 70 0B 06 97 0A 4B 10 4F 4B  26.d.{.p....K.OK
00E0    34 9F F6 34 FC 4E 18 7D 1B 52 76 05 99 4C 5B A9  4..4.N.}.Rv..L[.
00F0    FA 05 F1 DB AC 63 FD 26 08 E3 44 52 16 4C D6 1A  .....c.&..DR.L..
0100    0F A2 4D 20 73 F0 B3 04 7D E0 C4 AF 18 45 B3 28  ..M s...}....E.(
0110    F9 DF CA A6 4C 74 56 FC E9 4A 13 C2 75 A8 22 87  ....LtV..J..u.".
0120    C1 F4 31 55 76 10 9D AE C8 D8 A0 8D 17 96 E2 38  ..1Uv..........8
0130    1A 10 7B 43 F0 90 D9 0A 46 04 D7 0B D3 4B 29 C2  ..{C....F....K).
0140    F8 C4 6A 7F 0B 6F C1 01 EF 73 57 14 8D 5A 40 83  ..j..o...sW..Z@.
```

Result: Packet 498 is an HTTPS packet sent from the Mac to 104.18.32.47 on port 443. The lengths agree: 14 (Ethernet header) + 1440 (IP total length) = 1454 (frame), and 20 (IP header) + 32 (TCP header) + 1388 (payload) = 1440. All IP and TCP header fields are decoded as usual, but the payload looks random and the text column is mostly dots. This is expected: HTTPS payloads are encrypted with TLS. The payload begins with 17 03 03, which marks a TLS record carrying encrypted application data.

(g) Checking UDP port 443 using QUIC

```
MENU
1. Show all packets
2. Filter packets
3. Inspect one packet
4. Exit
Choose 1-4: 2

Enter a value to filter by, or press Enter to skip that filter.
Protocol (TCP/UDP): UDP
Source IP:
Destination IP:
Source port:
Destination port: 443

No.   Time          Source IP        Src Port  Destination IP   Dst Port  Protocol  Length
-------------------------------------------------------------------------------------------
192   05:05:47.376  192.168.40.117   57445     142.251.150.119  443       UDP       71
2138  05:06:16.379  192.168.40.117   57445     142.251.150.119  443       UDP       222
2879  05:06:29.678  192.168.40.117   59720     35.190.80.1      443       UDP       1292
2880  05:06:29.680  192.168.40.117   59720     35.190.80.1      443       UDP       1292
2892  05:06:29.697  192.168.40.117   59720     35.190.80.1      443       UDP       1292
2903  05:06:29.719  192.168.40.117   59720     35.190.80.1      443       UDP       83
2904  05:06:29.719  192.168.40.117   59720     35.190.80.1      443       UDP       83
2907  05:06:29.722  192.168.40.117   59720     35.190.80.1      443       UDP       214
2910  05:06:29.725  192.168.40.117   59720     35.190.80.1      443       UDP       75
2911  05:06:29.726  192.168.40.117   59720     35.190.80.1      443       UDP       75

10 packet(s) shown. Length = captured frame size in bytes. '-' = no port (not TCP/UDP, or header incomplete).
```

```
MENU
1. Show all packets
2. Filter packets
3. Inspect one packet
4. Exit
Choose 1-4: 3
Enter the packet number to inspect: 2879

==================================================
PACKET #2879
Time: 05:06:29.678
Packet length: 1292 bytes (captured frame)

IP HEADER
------------------------------
Version: 4
Header length: 20 bytes
Total length: 1278 bytes
TTL: 128
Protocol: UDP (number 17)
Header checksum: 0x7182
Source IP: 192.168.40.117
Destination IP: 35.190.80.1
Fragment offset: 0

UDP HEADER
------------------------------
Source port: 59720
Destination port: 443
Length: 1258 bytes
Checksum: 0xD8C0
Payload length: 1250 bytes

PAYLOAD
------------------------------
Payload length: 1250 bytes
Offset  Hex                                             Text
0000    C2 00 00 00 01 08 98 A9 68 6D FE 98 8E E5 00 00  ........hm......
0010    4U D0 3B 40 FE BD 2F 0F 25 B0 4E 7F 78 6A 70 6A  D;I../.%.N.xjyj
0020    E1 2F D9 A2 C9 67 BE 11 41 D9 78 80 9A 6E 6A 9E  ./...g..A.x..nj.
0030    BE 15 7B A6 B7 7A 37 40 2E 2A E7 17 1E 91 C7 C0  ..{..z7@.*......
0040    06 B0 1E EB C4 B3 21 26 C7 A4 86 15 03 D2 A5 E6  ......!&........
0050    92 01 7F 32 03 F6 15 7A 76 1E D3 9A 1F 9B 52 0B  ...2...zv.....R.
0060    4A D2 EE 91 5C 3D 31 34 8A 27 F0 42 26 9F 14 DB  J...\=14.'.B&...
0070    39 04 44 09 18 CE 0D D3 FF 2A 53 AA C0 E7 04 CA  9.D......*S.....
0080    38 80 03 E1 A1 C4 ED F4 5D A4 57 B1 A5 F8 5A 00  8.......].W...Z.
0090    91 DE F2 D7 57 95 47 DF 3A 0F BF 61 50 28 7A 1A  ....W.G.:..aP(z.
00A0    09 65 31 17 A6 7A C1 FE 21 B8 27 56 10 B8 0B C6  .e1..z..!.'V....
00B0    0C FC 59 8D C0 71 17 18 32 13 10 F0 CC BD CB BC  ..Y..q..2.......
00C0    40 41 D9 AC ED 9B C9 51 DD C9 A9 60 35 4D 54 DA  @A.....Q...`5MT.
00D0    F8 36 41 A3 71 5B 72 01 F2 F3 3E 2E DB 23 83 CC  .6A.qXr...>..#..
00E0    70 99 64 A9 41 7B F2 E8 2E C0 67 CA 34 E4 25 7B  p.d.A{....g.4.%{
00F0    BC 29 C6 3B AF 71 73 AD 67 E6 D3 3C 09 9D 69 92  .);.qs.g..<..i.
0100    1D D4 4A 18 76 DC D6 06 46 D1 EF D1 E4 1F BD 4B  ..J.v...F......H
0110    4B 44 EA DC 6F C4 23 B1 9E 7F 22 A9 6F 2F 00 E8  KD..o.#..."o/..
0120    38 04 7A BA 1F 49 B5 B7 E0 1F CB F4 66 A1 E0 73  8.z..I......f..s
0130    C7 AC AB 77 FC F9 47 86 1A 69 EF 73 DC 00 1C 71  ...w..G..i.s...q
0140    3E 5C 65 AC 9D 43 B9 09 77 91 FD 55 77 52 DE E2  >\e..C..w..UwR..
0150    30 F2 2E 20 03 43 CA 09 0D 9A 2F 7E 8F 9C 1B 44  0.. .C..../~...D
0160    CB ED A6 26 9B 6B 63 16 EF 2D 07 2E 43 D9 8F DA  ...&.kc..-..C...
0170    8D 75 36 1B 1F 43 5B AE 24 36 3E 47 AF FF 39 D0  .u6..C[.$6>G..9.
0180    41 2B E2 F2 EB 0E D9 87 F5 B5 2F 27 AF 10 6E 38  A+......../'..n8
```

Result: A UDP packet to port 443 (packet 2879). UDP length 1258 = 8 + 1250 (payload), and IP total length 1278 = 20 + 1258. QUIC (used by HTTP/3) encrypts its data, so the payload is unreadable, as expected.

7. Menu Error Handling and Program Exit Tests

(a) Testing an invalid menu choice

```
MENU
1. Show all packets
2. Filter packets
3. Inspect one packet
4. Exit
Choose 1-4: 7
Please choose 1, 2, 3, or 4.
```

(b) Testing Blank Menu Input

```
MENU
1. Show all packets
2. Filter packets
3. Inspect one packet
4. Exit
Choose 1-4:
Please choose 1, 2, 3, or 4.
```

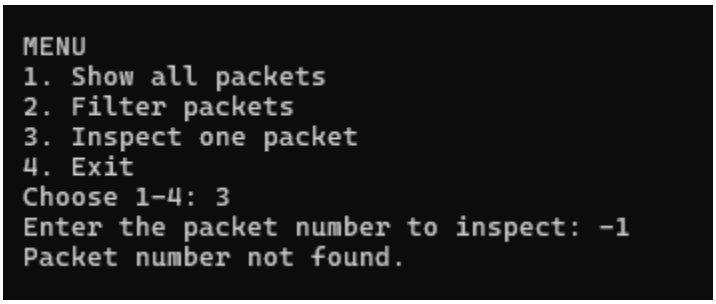
(c) Testing a Non-Numeric Packet Number

```
MENU
1. Show all packets
2. Filter packets
3. Inspect one packet
4. Exit
Choose 1-4: 3
Enter the packet number to inspect: abc
Please enter a whole number.
```

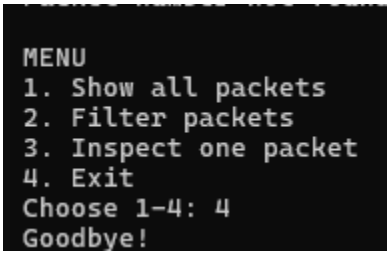
(d) Testing a Packet Number That Does Not Exist

```
MENU
1. Show all packets
2. Filter packets
3. Inspect one packet
4. Exit
Choose 1-4: 3
Enter the packet number to inspect: 999999
Packet number not found.
```

(e) Testing a Negative Packet Number



(f) Testing Program Exit



Result: Every invalid input produced a clear message and returned to the menu instead of crashing, and option 4 exited cleanly with "Goodbye!".

Challenges and fixes

- **Incoming packets missing on Windows.** During testing on Windows, the HTTP request appeared but the server's replies did not. Inspecting the raw bytes showed that incoming frames on that Wi-Fi network carried an 802.1Q VLAN tag (EtherType 0x8100), which the parser was skipping. The parser now skips the 4-byte tag and reads the real EtherType, so packets received by the host are decoded too.
- **Windows interface names.** On Windows, Scapy names interfaces like \Device\NPF_{GUID}, and the program defaulted to the macOS name en0. The program now lists interfaces by number with friendly names (such as "Wi-Fi") and picks the system's default interface.
- **Unclear filter results.** Previously, an invalid port or IP simply returned no packets, which looked like "no matches". Filter values are now validated and errors are reported.
- **Unreadable payloads.** Printing payload bytes directly could break the terminal output (for example with newline or escape characters). The payload is now shown as a hex dump with non-printable bytes displayed as ".".

Limitations

- Only Ethernet frames carrying IPv4 are decoded (with or without one VLAN tag). IPv6, double-tagged frames and later IP fragments are not fully decoded.

- Checksums are displayed but not verified.
- Protocol filtering supports TCP and UDP only, as the assignment specifies, and each IP or port filter matches one direction (source or destination).
- The Length column in the summary is the captured frame size (including the Ethernet header); the IP total length is shown in the detailed view.
- The payload text column shows ASCII only.

Privacy and authorization

All captures were made on our own computers, on a network we were authorized to use. Captured payloads can contain private information such as device names and local addresses, so no raw packet captures were committed to the repository.

Conclusion

We built a command-line packet analyzer that captures live traffic, decodes the IPv4, TCP and UDP headers directly from raw bytes, and lets the user filter packets and inspect each one in detail. All three offline test suites pass, and live tests on macOS and Windows confirmed every requirement of the assignment, including a full TCP handshake, a readable HTTP request and response, DNS queries over UDP, and encrypted HTTPS and QUIC traffic. Working at the byte level showed us how each header field maps to specific bytes and bits, and how real networks add details such as VLAN tags and TCP options that a parser has to handle.

AI Usage

The following prompts were used:

1. Explain me what exactly we need to implement in this assignment and guide me through making the repository.
2. Now that we have gone through the requirements and understood what to do, let’s do step-by-step designing of the Python-based Mini Packet Analyzer using Scapy.
3. Walk me through how to write code to capture network packets and manually parse Ethernet, IP, TCP, and UDP headers.
4. Then now let’s try to implement packet filtering based on protocol, IP addresses, and port numbers.
5. Create a display system to show packet summaries and detailed packet information, including hex and ASCII payloads."
6. Walk me through in adding input validation, error handling, and test cases for the packet analyzer.

Finally, we used it to help us debug and improve the code while maintaining the existing project structure.